Autoencoder

Hyoung Joon, Kim

June 3, 2026

Contents

1 Introduction

Autoencoder produces outputs that are similar to input data. It first maps the input x to a hidden representation $z(x)$ and produce the output $y(z)$ which reserves the original data x well. This can be seen as two parts, *encoder* and *decoder*. Encoder can later be used at dimensionality reduction. To inhibit autoendcoder from generating trivial solution, we give some constraints, such as restricting the dimension of hidden representation or make it to have sparse representation.

2 Deterministic Autoencoder

2.1 Deep Autoencoder

Definition 2.1.1 (Deep Autoencoder)

Let $\{\mathbf{x}_n\} \in \mathbb{R}^D$ be a given data, and let $\mathbf{y}(\mathbf{x}, \mathbf{w}) : \mathbb{R}^D \to \mathbb{R}^D$ be a deep network function with parameter $\mathbf{w}$ having bottleneck. Then, the **deep autoencoder** minizes the reconstruction error:

$$E(\mathbf{w}) = \sum_{n=1}^{N} \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{x}_n\|^2$$

Usually the network $\mathbf{y}(\mathbf{x}, \mathbf{w})$ decrease the dimensionality and then increase dimensionality of hidden units to stop autoencoder from learning trivial representations, which is called bottleneck.

2.2 Sparse Autoencoder

Instead of making bottleneck like deep autoencoder, we can use regularizer so that the internal representation have a sparse representation. This method have an effect of inducing the effective dimensionality of internal representation. One simple choice of sparse regularizer is L1 regularizer,

Definition 2.2.1 (Sparse Autoencoder with L1 Regularizer)

Let $\{\mathbf{x}_n\} \in \mathbb{R}^D$ be a given data, and let $\mathbf{y}(\mathbf{x}, \mathbf{w}) : \mathbb{R}^D \to \mathbb{R}^D$ be a deep network function with parameter $\mathbf{w}$. Then, the **sparse autoencoder** minimizes the reconstruction error with L1 regularizer:

$$E(\mathbf{w}) = \sum_{n=1}^{N} \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{x}_n\|^2 + \lambda \sum_{k=1}^{K} |z_k|$$

where z_k are the appropriate hidden units according to the internal representation.

2.3 Denoising Autoencoder

Another constraint is to corrupt the given data. Then the autoencoder will have ability to denoise the corrupted data.

Definition 2.3.1 (Denoising Autoencoder)

Let $\{\mathbf{x}_n\} \in \mathbb{R}^D$ be a given data, and let $\mathbf{y}(\mathbf{x}, \mathbf{w}) : \mathbb{R}^D \to \mathbb{R}^D$ be a deep network function with parameter $\mathbf{w}$. Then, the **denoising autoencoder** minizes the following error function:

$$E(\mathbf{w}) = \sum_{n=1}^{N} \|\mathbf{y}(\tilde{\mathbf{x}}_n, \mathbf{w}) - \mathbf{x}_n\|^2$$

where $\tilde{\mathbf{x}}_n$ are corrupted dataset.

The denoising autoencoder's training is related to score mathcing where the score is defined as $\mathbf{s} = \nabla_{\mathbf{x}} \log p(\mathbf{x})$. The autoencoder learns to reverse the corruption, and the score vector $\nabla \log p(x)$ points towards high data density, These two methods are similar and have deep relationship.

2.4 Masked Autoencoder

The transformer models such as BERT are trained by masking the random subsets of input texts, and this can be applied to autoencoder models, too. This is usually combined with transformer models to create a vision transformer architecture. Then the autoencoder trains to predict the deleted pixels, inferencing with leftover pixels.

3 Variational Autoencoder

3.1 Introduction

The **variational autoencoder**, unlike deterministic autoencoder, does not fix the internal representation, but allow some uncertainty of internal representation. However, the likelihood defined by a neural network make the training and inference intractable. Variational autoencoder resolve this problem by using *variational inference* technique, approximating the likelihood function with a feasible function.

3.2 Amortized Inference

Consider a generative model with latent variable having distribution $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I})$. We can make a network that creates data from latent variables: $\mathbf{g}(\mathbf{z}, \mathbf{w})$. And we can model a probability distribution with network $\mathbf{g}$, for example, a Gaussian distribution $p(\mathbf{x}|\mathbf{z}, \mathbf{w}) = \mathcal{N}(\mathbf{x}|\mathbf{g}(\mathbf{z}, \mathbf{w}), \mathbf{I})$.

Now we derive the ELBO of variational autoencoder. The log-likelihood of data $\mathbf{x}$ is:

$$\log p(\mathbf{x}|\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \mathrm{KL}(q(\mathbf{z})\|p(\mathbf{z}|\mathbf{x}, \mathbf{w}))$$

where $\mathcal{L}(\mathbf{w})$ is the evidence lower bound,

$$\mathcal{L}(\mathbf{w}) = \int q(\mathbf{z}) \log \left\{ \frac{p(\mathbf{x}|\mathbf{z}, \mathbf{w})p(\mathbf{z})}{q(\mathbf{z})} \right\} d\mathbf{z}$$

and

$$\mathrm{KL}(q(\mathbf{z})\|p(\mathbf{z}|\mathbf{x}, \mathbf{w})) = -\int q(\mathbf{z}) \log \left\{ \frac{p(\mathbf{z}|\mathbf{x}, \mathbf{w})}{q(\mathbf{z})} \right\} d\mathbf{z}.$$

The difference from GMM or pPCA is that the E step is infeasible, since the exact posterior distribution requires the likelihood $p(\mathbf{x}|\mathbf{w})$, which is intractable:

$$p(\mathbf{z}|\mathbf{x}, \mathbf{w}) = \frac{p(\mathbf{x}|\mathbf{z}, \mathbf{w})p(\mathbf{z})}{p(\mathbf{x}|\mathbf{w})}$$

The solution is to approximate the posterior distribution $p(\mathbf{z}|\mathbf{x}, \mathbf{w})$, with another network $q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi})$, called the *encoder network*. The original network then can be called as *decoder network*.

A typical choice for the encoder network is a Guassian ditribution with mean and variance given by the function that takes $\mathbf{x}$ as input;

$$q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) = \prod_{j=1}^{M} \mathcal{N}(z_j|\mu_j(\mathbf{x}, \boldsymbol{\phi}), \sigma_j^2(\mathbf{x}, \boldsymbol{\phi}))$$

The E step now coressponds to updating $\boldsymbol{\phi}$ and M step corresponds to updating $\mathbf{w}$. Notice that the E step does not in general reduce the KL divergence to zero, since it is just an approximation.

3.3 Reparametrization Trick

Now, we introduce the **reparametrization trick**. The lower bound $\mathcal{L}(\mathbf{w})$ is still intractable since it involves integrals over the latent variables $\{\mathbf{z}_n\}$:

$$\begin{aligned}\mathcal{L}(\mathbf{w}, \boldsymbol{\phi}) &= \int q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) \log \left\{ \frac{p(\mathbf{x}|\mathbf{z}, \mathbf{w})p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi})} \right\} d\mathbf{z} \\ &= \int q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) \log p(\mathbf{x}|\mathbf{z}, \mathbf{w}) d\mathbf{z} - \mathrm{KL}(q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi})\|p(\mathbf{z}))\end{aligned}$$

The first term is called *reconstruction error*, since it can be seen as expectation of likelihood of x about latent variable created from given data x. The second term, is called regularization term, and can be evaluated analytically:

$$\mathrm{KL}(q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi})\|p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^{M} \{1 + \log \sigma_j^2(\mathbf{x}, \boldsymbol{\phi}) - \mu_j^2(\mathbf{x}, \boldsymbol{\phi}) - \sigma_j^2(\mathbf{x}, \boldsymbol{\phi})\}$$

The leftover term can be approximated by simple Monte Carlo estimator:

$$\int q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) \log p(\mathbf{x}|\mathbf{z}, \mathbf{w}) d\mathbf{z} \simeq \frac{1}{L} \sum_{l=1}^{L} \log p(\mathbf{x}|\mathbf{z}^{(l)}, \mathbf{w})$$

where $\{\mathbf{z}^{(l)}\}$ are samples drawn from the encoder distribution $q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi})$. The problem is that the samples are fixed. The change in parameter $\boldsymbol{\phi}$ will change the distribution where the samples are sampled from, but the sample stays fixed. To resolve this, we use reparametrization trick. Instead of sampling directly from $q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi})$, we use that $\mathbf{z}_j = \sigma_j \epsilon + \mu_j$ where $\epsilon \sim \mathcal{N}(0, 1)$ has Guassian distribution with mean μ_j and variance σ_j^2, and sample $\epsilon \sim \mathcal{N}(0, 1)$ to create samples $\mathbf{z}^{(l)}$. This makes the dependence on $\boldsymbol{\phi}$ explicit and allow us to use gradient descent methods.

3.4 Algorithm

The training of VAE must be balanced over two terms, the reconstruction error and KL divergence.

Algorithm 1: Variational autoencoder training algorithm

Input : dataset $\mathcal{D} = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$
encoder network $q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) = \prod_{j=1}^{M} \mathcal{N}(z_j|\mu_j(\mathbf{x}, \boldsymbol{\phi}), \sigma_j^2(\mathbf{x}, \boldsymbol{\phi}))$
decoder network $\mathbf{g}(\mathbf{z}, \mathbf{w})$
initial weight vectors $\mathbf{w}$, $\boldsymbol{\phi}$
learning rate η

Output: final weight vectors $\mathbf{w}$, $\boldsymbol{\phi}$

repeat
  for $n \in$ mini-batches of data **do**
    $\mathcal{L} \leftarrow 0$
    for $j \in \{1, \ldots, M\}$ **do**
      $\epsilon_{nj} \leftarrow \mathcal{N}(0, 1)$
      $z_{nj} \leftarrow \mu_j(\mathbf{x}_n, \boldsymbol{\phi}) + \sigma_j(\mathbf{x}_n, \boldsymbol{\phi})\epsilon_{nj}$
      $\mathcal{L} \leftarrow \mathcal{L} + \frac{1}{2}\{1 + \log \sigma_j^2(\mathbf{x}_n, \boldsymbol{\phi}) - \mu_j^2(\mathbf{x}_n, \boldsymbol{\phi}) - \sigma_j^2(\mathbf{x}_n, \boldsymbol{\phi})\}$
    $\mathcal{L} \leftarrow \mathcal{L} + \log p(\mathbf{x}_n|\mathbf{z}_n, \mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \eta \nabla_{\mathbf{w}} \mathcal{L}$
    $\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} + \eta \nabla_{\boldsymbol{\phi}} \mathcal{L}$
until convergence
return $\mathbf{w}$, $\boldsymbol{\phi}$

The decoder network $\mathbf{g}(\mathbf{z}, \mathbf{w})$ is inherent in $p(\mathbf{x}|\mathbf{z}, \mathbf{w})$. For example, when the variables are continuous, $p(\mathbf{x}|\mathbf{z}, \mathbf{w}) = \mathcal{N}(\mathbf{g}(\mathbf{z}, \mathbf{w}), \mathbf{I})$

3.4.1 Posterior Collapse and Holes in Latent Space

The VAE training algorithm, Algorithm 1 has a problem called **posterior collapse**. When the variational distribution $q(\mathbf{z}|\mathbf{x}, \mathbf{w})$ converges to the prior distribution $p(z)$ and ignores $\mathbf{x}$, leading to poor reconstruction result. This happens because the second term in equation

$$\begin{aligned}\mathcal{L}(\mathbf{w}, \boldsymbol{\phi}) &= \int q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) \log \left\{ \frac{p(\mathbf{x}|\mathbf{z}, \mathbf{w})p(\mathbf{z})}{q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi})} \right\} d\mathbf{z} \\ &= \int q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) \log p(\mathbf{x}|\mathbf{z}, \mathbf{w}) d\mathbf{z} - \mathrm{KL}(q(\mathbf{z}|\mathbf{x}, \boldsymbol{\phi}) \| p(\mathbf{z})),\end{aligned}$$

the KL divergence term, encourage $q(\mathbf{z}|\mathbf{x}, \mathbf{w})$ to be close to the prior $p(\mathbf{z})$. Thus the posterior collapse can happen when the KL divergence term is relatively closer to zero than the reconstruction term.

Different problem can happen when the latent code is not compressed well. When the latent code is not compressed and therefore is very different from prior, the reconstruction result may be highly accurate, but the samples from prior won't generate realistic outupts. We say that there are **holes in latent space**. In this case, the KL divergence term would be relatively larger than the reconstruction error.

3.4.2 Solution

We can address both problems by multiplying a coeffiecient β in front of reconstruction term, to control the regularization effectiveness of KL divergence term. Typically, $\beta > 1$. If the reconstruction is poor, we increse β. When the samples are poor, we decrease β. Or we can change β during training session, by starting with a small value and gradually increasing.

Another method is to modify the KL divergence term. Wasserstein Autoencoder, VQ-VAE and InfoVAE is the examples of it.

Guess I should study these methods when I build VAE from scratch...